DEPLOY SURE AI

A Multi-Agent System for Intelligent Software Release Risk Assessment and Deployment Validation

Should we deploy this release? Let the agents decide.

Parth Koshti(24070126093)
Krrish Talesara(24070126096

The Problem

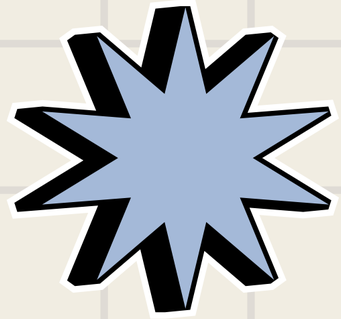
Software releases are risky, and the decision to ship is still mostly manual.

The scenario

A team ships v2.1 with changes to payment, auth, and notifications.
Before merging to production, a developer has to answer one question:

"Is this release actually safe to deploy?"

Answering it requires reading diffs, tracing dependencies, selecting tests, interpreting failures, and judging severity. All manual, every release.



What a bad release costs

- Customer-facing failures
- Payment and data errors
- Downtime and emergency rollbacks
- Reputational damage
- Developer time lost firefighting

Why This Needs Agentic AI

Normal chatbot

✗Returns text, not actions

✗Cannot run the test suite

✗Cannot trace code dependencies

✗No way to validate its own answer

✗No memory of multi-step reasoning

Agentic AI system

✓Planner decomposes the release goal

✓Specialist agents reason over each slice

✓Executor runs real pytest via tool calls

✓Critic validates the decision independently

✓LangGraph routes retries and branches

Proposed Solution: DeploySure AI

Analyzes a release → runs only the relevant tests → scores risk → decides Deploy / Hold / Block → a Critic agent validates the decision

DEPLOY

All critical tests pass. Risk is low. Ship it.

HOLD

Non-critical issues or inconclusive evidence. Manual review.

BLOCK

Critical tests fail or risk is too high. Do not ship.

What sets it apart • self-validating via a Critic agent with a bounded retry loop

Agents – Structure

Each agent has a specific role, input, tools and output in the DeploySure AI workflow.

01 Planner Agent

Plans the release validation workflow

Role
- Understand release context
- Break down tasks
- Coordinate next steps

Inputs
- Release version (v2.1)
- Previous version (v2.0)
- Release notes

Tools
- LLM (reasoning)
- Project file reader

Output
- Validation plan and task list

Example Output

1. Analyze code changes
2. Identify impacted modules
3. Select and run relevant tests
4. Assess risk and decide

02 Change Analysis Agent

Analyzes code changes in the new release

Role
- Find what changed
- Classify change type
- Extract affected functions

Inputs
- Previous version
- Current version
- Project codebase

Tools
- Git/Diff tool
- Code parser (AST)
- File search

Output
- Changed files, functions and change summary

Example Output

Changed files:
- payment.py (logic change)
- auth.py (validation change)

Changed functions:
- process_payment()
- validate_user()

03 Impact Analysis Agent

Identifies affected modules and dependencies

Role
- Find modules impacted by the changes
- Analyze dependencies
- Determine impact level

Inputs
- Changed files/functions
- Project codebase
- Dependency graph

Tools
- Dependency analyzer
- Code search
- Graph analysis

Output
- Impacted modules with impact level (High/Medium/Low)

Example Output

Impacted modules:
- Payment – High
- Checkout – High
- Orders – Medium
- Notification – Low

04 Test Selection Agent

Selects relevant test cases to run

Role
- Choose only necessary tests
- Prioritize tests based on impact
- Reduce unnecessary runs

Inputs
- Impacted modules
- Available test suite
- Change summary

Tools
- Test mapping logic
- File search (pytest files)
- LLM (test selection reasoning)

Output
- List of selected test cases

Example Output

Selected tests:
- test_payment_success
- test_payment_failure
- test_checkout_payment
- test_order_creation
- test_auth_login

Agents – Structure

Each agent has a specific role, input, tools and output in the DeploySure AI workflow.

05 Test Execution Agent

Runs the selected tests and collects results

Role
- Execute selected tests
- Capture test results
- Identify failures and logs

Inputs
- Selected test cases
- Test suite
- Test environment (project code)

Tools
- Pytest runner
- Subprocess execution
- Log parser

Output
- Test results (pass/fail)
- Error logs for failed tests

Example Output

Total tests: 6
Passed: 4
Failed: 2
Failed tests:
- test_refund.py
- test_payment_timeout.py

06 Risk Assessment Agent

Evaluates failures and calculates release risk

Role
- Analyze test results
- Consider component criticality
- Calculate risk score and level

Inputs
- Test results
- Impacted modules
- Change severity summary

Tools
- Risk scoring logic
- Component criticality map
- LLM reasoning (explain risk)

Output
- Risk score (0–100)
- Risk level (Low/Medium/High)
- Risk factors and reasoning

Example Output

Risk Score: 82 / 100
Risk Level: HIGH
Key Factors:
- Payment module failure (Critical)
- Customer-facing feature affected
- 2 important tests failed

07 Decision Agent

Recommends final deployment decision

Role
- Interpret risk assessment
- Consider business rules
- Recommend DEPLOY / HOLD / BLOCK

Inputs
- Risk score and level
- Test results summary
- Change and impact analysis

Tools
- Decision rules
- LLM reasoning (explain decision)

Output
- Recommended decision
- Decision reasoning
- Confidence level

Example Output

Decision: BLOCK
Reason: Critical payment tests failed and risk score is high.
Confidence: 0.87
Next Step: Fix issues and re-test.

08 Critic Agent

Validates decision and checks for gaps

Role
- Review decision against evidence
- Check for missing risks or tests
- Approve or suggest re-evaluation

Inputs
- Decision and reasoning
- Risk assessment
- All previous agent outputs

Tools
- Consistency checker
- Evidence verification (logs, tests)
- LLM reasoning

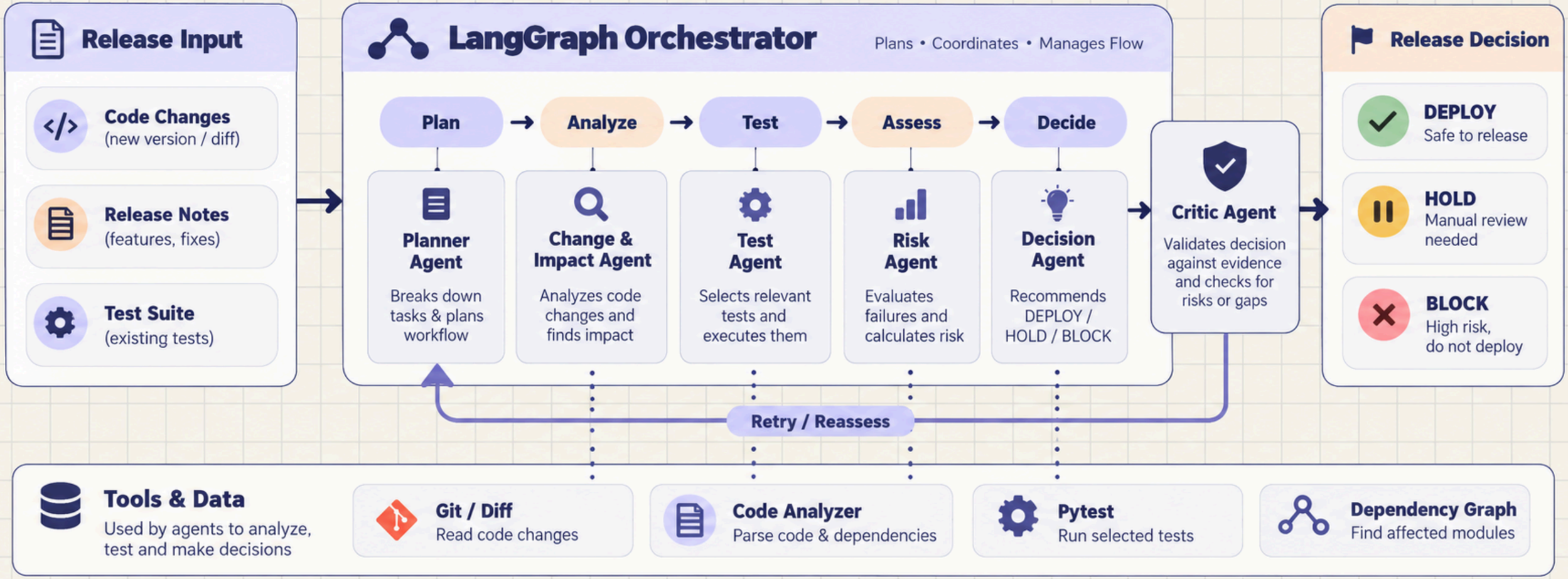
Output
- Validation result (Approve/Reject)
- Comments and suggested actions
- Trigger retry if needed

Example Output

Validation: REJECT
Reason: A critical test failure was overlooked in the decision.
Action: Reassess with additional tests.
Retry: Yes (re-plan workflow)

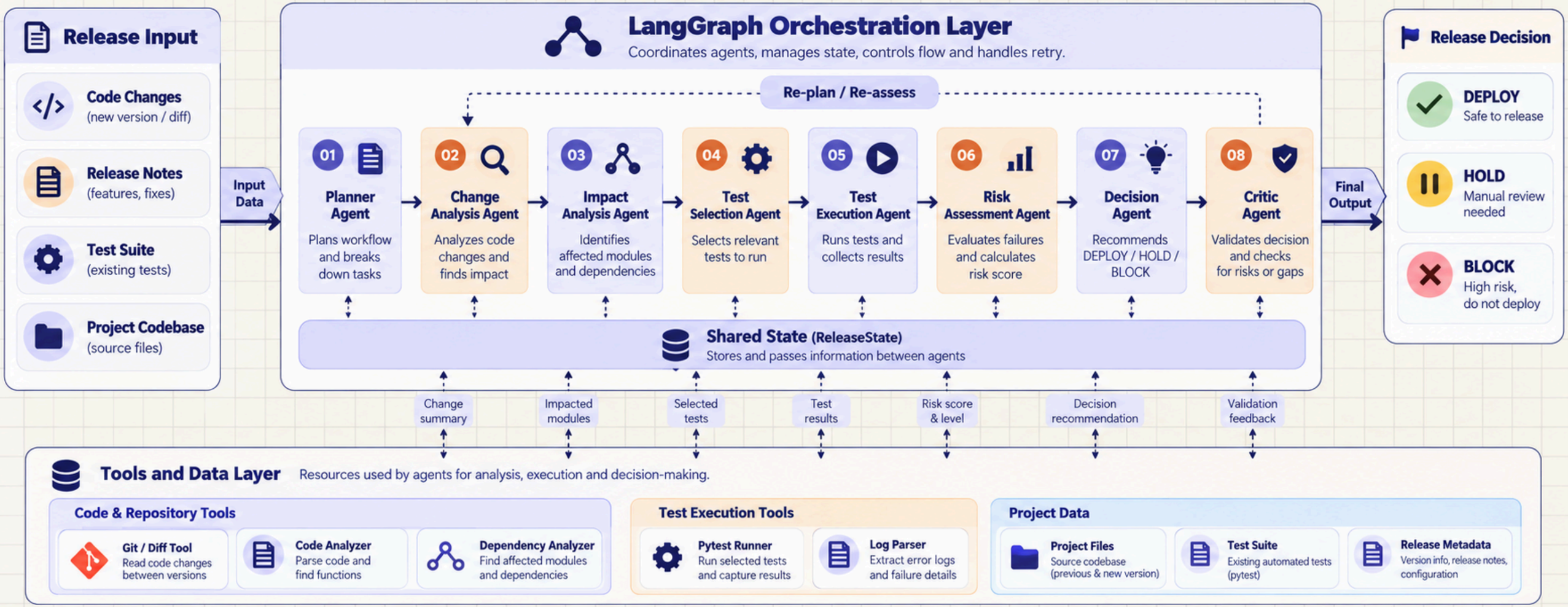
DeploySure AI — System Architecture

Multi-Agent Software Release Risk Assessment & Validation



DeploySure AI
Orchestration and Data Flow

How agents, tools and data interact to analyze a software release and make a validated deployment decision.



Scenario A · Safe Release

A low-risk change that the system clears for deployment.

CHANGE
notification.py
cosmetic update

IMPACT
LOW
isolated module

TESTS SELECTED
6 selected
notification suite

EXECUTION
6 / 6 PASS
zero failures

RISK
12 / 100
LOW severity

Decision · DEPLOY

All critical tests passed, risk is low, impact is isolated. Release is safe to ship.

Critic ✓ Validated

Decision is consistent with the evidence.
No reassessment required.

Scenario B · Dangerous Release — the Critic Loop in Action

Everything looks shippable until the Critic sees the evidence.

CHANGE
payment.py
process_payment() logic edited

→

IMPACT
HIGH
payment, checkout, orders

→

EXECUTION
18 / 20
test_refund FAIL

→

RISK
87 / 100
CRITICAL

Decision Agent · DEPLOY
(initial recommendation, not final)

→

Critic REJECTS
Critical payment test failed. Payment is customer-facing.
Risk score is 87. Decision is inconsistent with the evidence.

▼ *reassessment loop* ▼

Final Decision · BLOCK RELEASE

Reassessment agrees with the Critic. v2.1 is blocked until the payment regression is fixed.

Stack · Validation · Limitations

What it is built on, how it checks itself, and what it does not claim.

Technology Stack

Python
core language

LangGraph
agent orchestration

Ollama
local LLM reasoning

pytest + subprocess
real test execution

difflib + AST
diff and impact

Custom UI
built separately

Output Validation

- Critic agent cross-checks every decision against the evidence
- Evidence-based risk scoring, not opinion
- Real pytest results, not LLM-fabricated outputs
- Bounded retry loop prevents infinite replanning
- Deterministic nodes for diff, execution, scoring

Limitations

- Synthetic demo codebase, not production scale
- Limited test coverage in the demo project
- Local-model reasoning can be imperfect
- Not a security scanner, no CVE detection
- Does not cover infrastructure or runtime factors
- Does not guarantee production safety, only risk assessment

Future Scope

Where DeploySure AI goes beyond the CA-3 proof of concept.

1. **GitHub + GitLab**
   Pull-request level validation on every PR.

2. **CI / CD integration**
   Jenkins, GitHub Actions, GitLab Pipelines.

3. **Docker environments**
   Spin up ephemeral containers per release.

4. **Security scanning**
   SAST, dependency CVE checks, secret detection.

5. **Auto rollback**
   Monitor post-deploy and roll back on regression.

6. **Historical learning**
   Learn risk patterns from past deployments.

Thank you